

Software Architecture- 03

KALINGA GUNAWARDHANA

DEPT OF COMPUTING & INFORMATION SYSTEMS

SABARAGAMUWA UNIVERSITY OF SRI LANKA

A solid green horizontal bar at the bottom of the slide.

Activity Diagrams

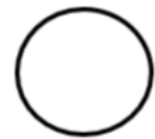
A **UML activity diagram** allows the representation of the control flow from activity to another in a software system. It captures the dynamic behaviour of the system and allows the mapping of branching into alternative flows.

Activities are actions that further the flow of execution in a system. They are actions that when completed cause another action to execute. For example, an action can alter or create new objects. These changes or actions can drive your application forward.

In order to create an activity diagram, you must:

1. Identify the activities.
2. Identify the respective conditions of the systems' activities.

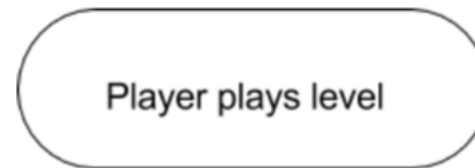
Once activities and respective conditions have been determined, the activity diagram can be created.



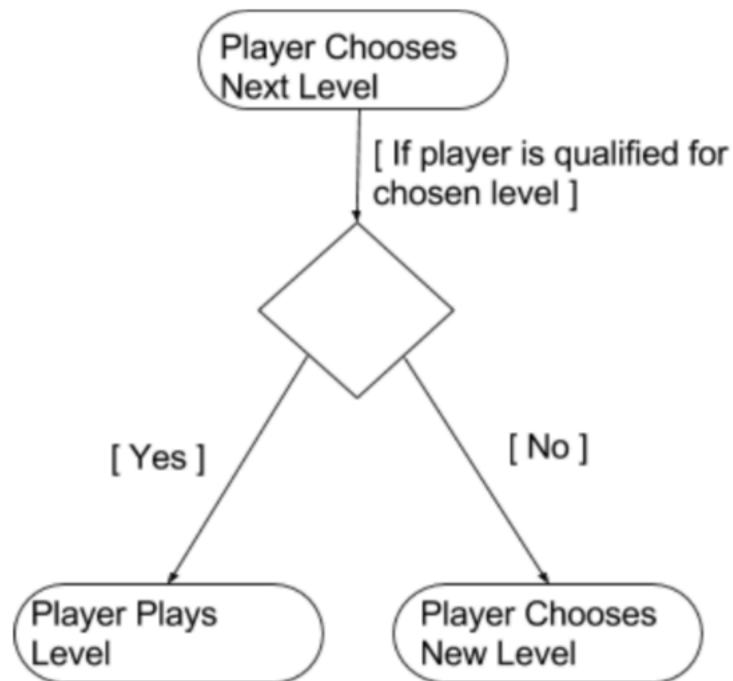
Start Game



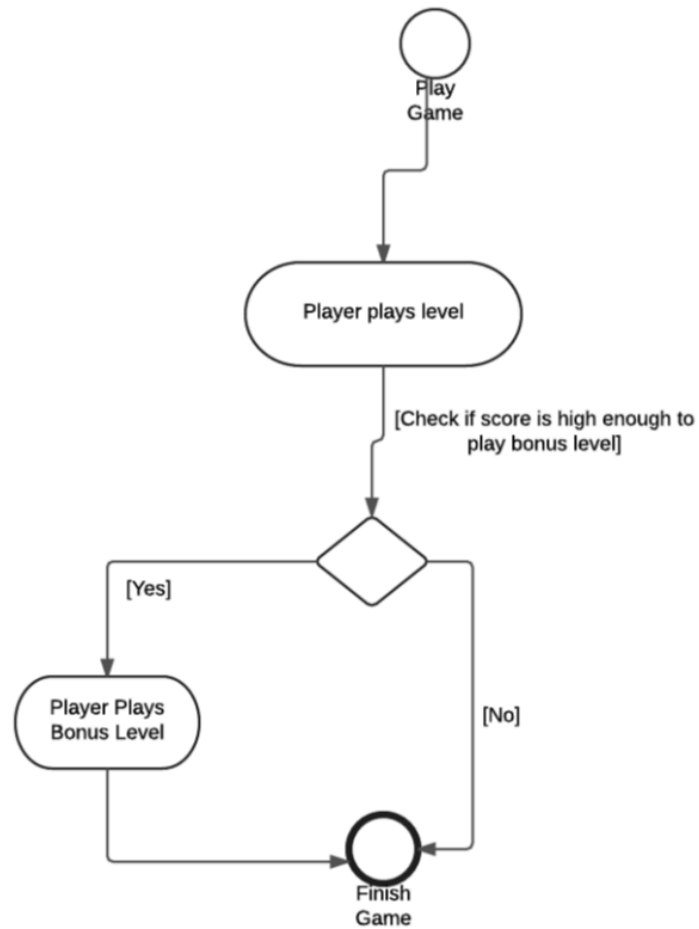
End Game



Additionally, activity diagrams can include decision nodes. These are diamonds that have an activity leading into it, and there is the possibility of two alternative outcomes as the next activity.



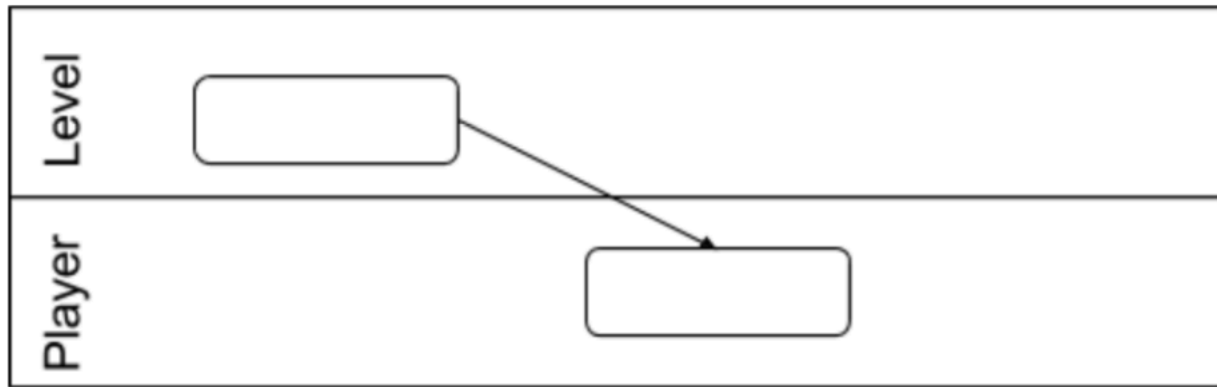
Ex: A video game



Activity diagrams allow the mapping of concurrent activities that happen in parallel. A fork in the flow moves into parallel flows, so activities can happen at the same time.

Parallel flows can join into a single flow. For example, if a level ends in a video game, the soundtrack flow will end at the same time as the play flow.

Swimlanes



Architectural Styles

1. Language-based systems
2. Repository-based systems
3. Layered systems
4. Interpreter-based systems
5. Dataflow systems
6. Implicit invocation systems
7. Process control systems

Language-Based Systems

The programming paradigm of the language selected to implement a system will affect the architectural style of that system. Each programming paradigm has its own set of constructs, principles, and design patterns, and their use shapes the system being created.

Object-oriented principles

Abstraction

*"Eliminate the Irrelevant,
Amplify the Essential"*

Encapsulation

"Hiding the Unnecessary"

Inheritance

"Modeling the Similarity"

Polymorphism

"Same Function Different Behavior"

Decomposition that breaks a whole into parts.

Generalization that allows you to factor out conceptual commonalities.

OO Design Patterns

Creational Patterns

Structural Patterns

Behavioral patterns

Quiz

Briefly provide examples with coding for the above patterns.